Robot Rush

2D Side Scrolling Action Survival Game

Built with Unity • Genre: Action Survival • Platform: Windows

SUBMISSION DETAILS	
Game Title	Robot Rush
Student Name	Dassanayake B.A.
IT Number	IT22178954
Module	SE4031 – Games Development
Assignment	Assignment 02 (Individual) – 20%
Submitted To	Mr. Aruna Ishara Gamage / Mr. Nushkan Nismi
Date	March 2026

Sri Lanka Institute of Information Technology

Faculty of Computing | Department of Software Engineering

Game Documentation | Confidential – Submission Copy

1. Game Overview

Robot Rush is a fast paced **2D Side Scrolling Action Survival Game** developed in **Unity** for the SE4031 Games Development module. The player controls a soldier character who must fight off waves of incoming robot enemies, dodge obstacles on a scrolling road, and collect crystal health packs to restore lost hearts surviving as long as possible against an ever accelerating threat.

Attribute	Detail
Game Title	Robot Rush
Genre	2D Side Scrolling Action Survival
Engine	Unity (2D)
Platform	Windows (.exe)
Student Name	Dassanayake B.A.
IT Number	IT22178954
Module	SE4031 – Games Development
Assignment	Assignment 02 (Individual) – 20%
Date	March 2026

2. Game Concept & Gameplay Summary

The core gameplay loop of Robot Rush tasks the player with navigating vertically (up and down) across a 2D road plane to dodge incoming robot enemies and obstacles, fire projectiles to eliminate robots, and collect orange crystal health packs to restore lost HP hearts. As time progresses, the **global speed multiplier** increases, accelerating both enemy spawning and road scroll speed escalating difficulty organically throughout the run.

The game features a **heart based health system** (3 hearts maximum, visible as a ❤❤❤ UI bar in the top left), a **real time score counter** in the top right that increments on enemy kills, and a **main menu** with Play and Quit buttons that also tracks a **High Score** (125,000 max observed during testing).

Table 1 Gameplay Loop Summary

Phase	Player Action	Outcome
Dodge Enemy	Move vertically to avoid	Avoid losing a heart / stay alive
Shoot Enemy	Left click to fire bullet	Destroy robot enemy (+1 score)
Health Pack	Collide with crystal pack	Restore 1 HP heart (up to max 3)
Time Passes	Survive longer	Speed multiplier increases globally

3. Gameplay Screenshots

The following screenshots were captured from the playable Windows build of Robot Rush, demonstrating the main menu and active gameplay session.

3.1 Main Menu Screen

Figure: *Robot Rush main menu title logo, Play/Quit buttons, and High Score display (125,000). Jungle arena background with soldier and spider robot characters.*

3.2 In Game Gameplay Screen

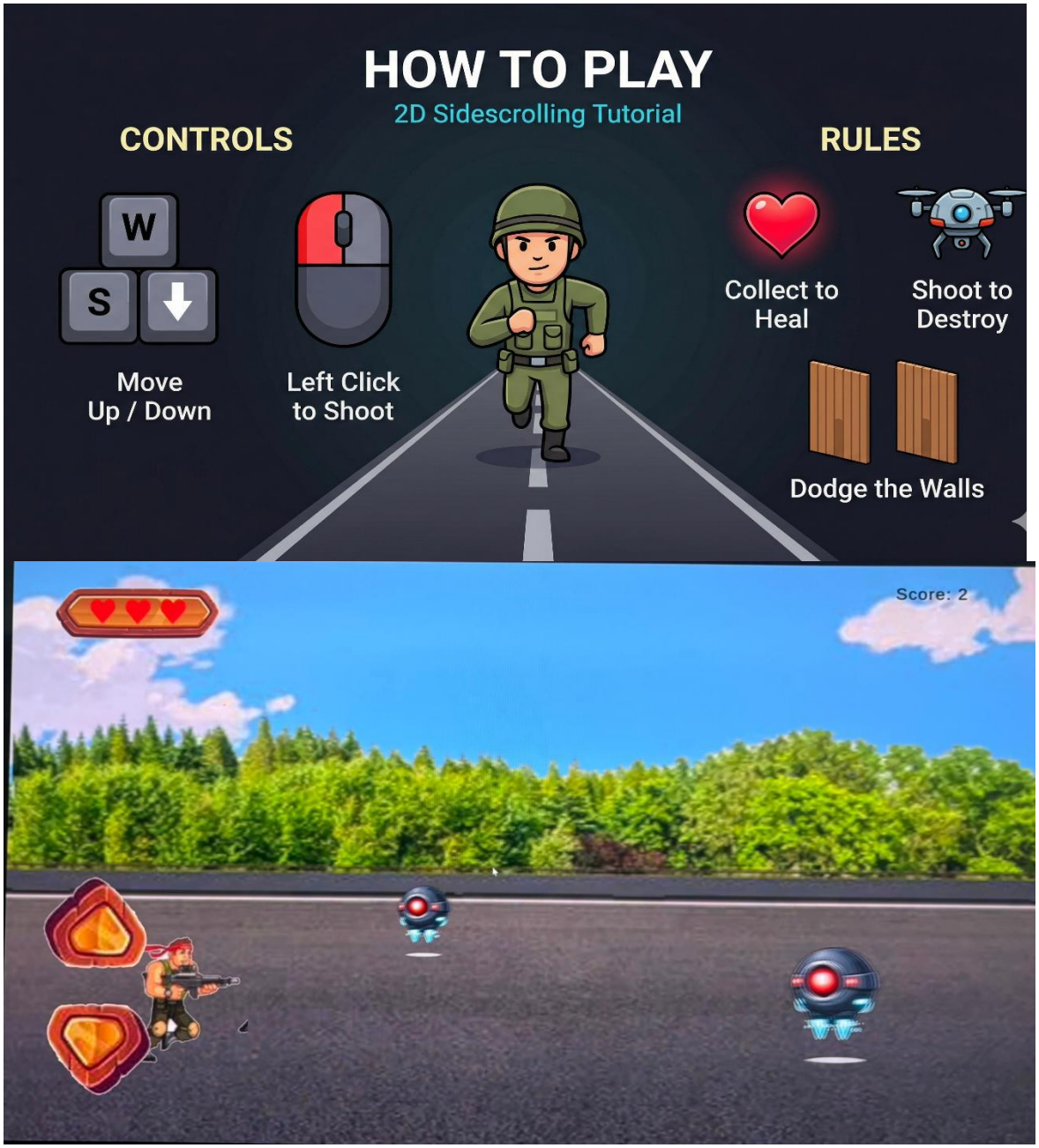


Figure: *Active gameplay heart based HP bar (top left, 3 hearts), Score counter (top right), soldier character, two robot enemies, and two crystal health packs on the scrolling road.*

4. Control Guide

Robot Rush uses keyboard input for vertical movement and mouse input for projectile attacks. All controls are mapped as follows:

Table 2 Control Mapping

Input	Action	Notes
W / ↑	Move Character Up	Vertical movement on 2D ground plane
S / ↓	Move Character Down	Vertical movement on 2D ground plane
Left Click	Fire Projectile	Spawns bullet at firePoint; destroys on contact or timeout

5. Technical Architecture & System Design

Robot Rush is built on Unity's **component based architecture**, where each game object carries focused, single responsibility scripts. The **GameManager** acts as a central **Singleton**, holding all global state (score, heart HP count, speedMultiplier) and providing a shared reference point for all other systems.

5.1 High Level Game Loop Diagram

The four primary systems interact in a unidirectional pipeline. The speed multiplier feedback loop is the key coupling mechanism driving progressive difficulty.

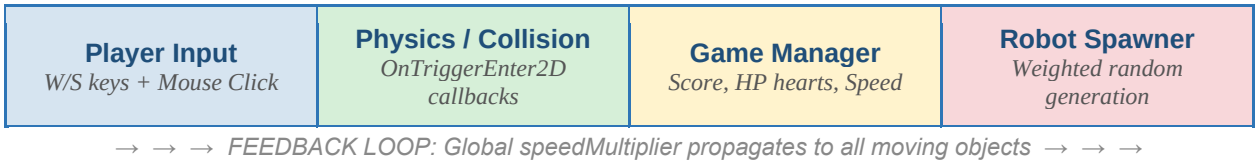


Figure: *High level game loop four systems connected by the global speed multiplier feedback loop.*

5.2 Core Scripts & Responsibilities

Table 3 Script Component Responsibility Matrix

Script	Attached To	Responsibility
GameManager.cs	GameManager Singleton	Global state, score, hearts HP, speedMultiplier, game over logic
PlayerController.cs	Player (Soldier)	Input handling, vertical movement, collision callbacks, shooting
Bullet.cs	Bullet Prefab	Forward movement, self destruction on collision or lifetime timeout
Spawner.cs	Spawner GameObject	Weighted random spawning: robots, crystal health packs
Obstacle.cs	Obstacle / Road Prefab	Left scroll movement referencing global speedMultiplier

HealthSystem.cs	Player / UI Canvas	Heart HP tracking, UI update, damage flash overlay, camera shake
Enemy.cs	Robot Enemy Prefab	Movement, collision with player or bullet, score award on death

6. Core Gameplay Mechanisms

6.1 Health & Damage System (Heart UI)

Robot Rush uses a **heart based health system** three hearts displayed as a ♥♥♥ icon bar in the top left of the screen (visible in Screenshot 3.2). The system is driven by Unity's `OnTriggerEnter2D` physics callback. When the soldier's collider intersects an object tagged ***Enemy*** or ***Obstacle***, the `TakeDamage(int damage)` method is invoked removing one heart, triggering visual feedback (damage flash + camera shake), and calling Game Over when all three hearts are lost.

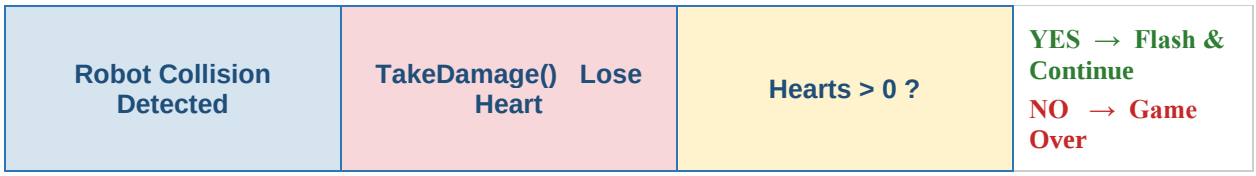


Figure: *Health & Damage system collision through to Game Over decision.*

Listing 1 PlayerController.cs: Collision Detection & Score Zone

```
void OnTriggerEnter2D(Collider2D other)
{
    if (other.CompareTag("Enemy") || other.CompareTag("Obstacle"))
    {
        TakeDamage(1);
        if (other.CompareTag("Enemy"))
        {
            Destroy(other.gameObject);  // Destroy robot on contact
        }
    }
    else if (other.CompareTag("ScoreZone"))
    {
        GameManager.instance.AddScore(1);
        other.gameObject.SetActive(false);  // Prevent double scoring
    }
}
```

6.2 Projectile Attack System

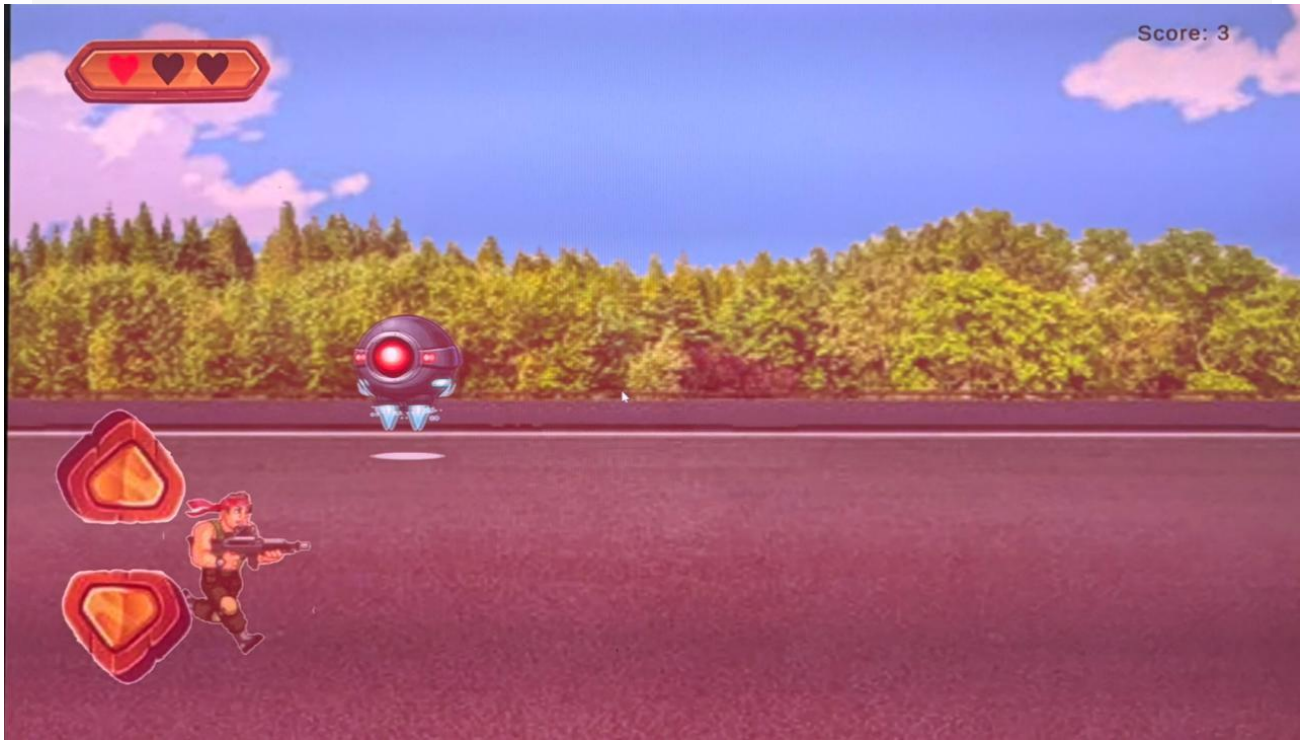
On left mouse click, the game **instantiates** a Bullet Prefab at the `firePoint` transform on the soldier character. The bullet uses `Time.deltaTime` for frame rate independent rightward movement and auto destroys on: (a) collision with any robot collider (awarding +1 score), or (b) expiry of a fixed lifetime timer.

Listing 2 Bullet.cs: Movement & Auto Destruction

```
// Bullet.cs   Movement & Auto Destroy
void Update()
{
```

```
    transform.Translate(Vector2.right * bulletSpeed * Time.deltaTime);
}

void OnTriggerEnter2D(Collider2D other)
{
    if (other.CompareTag("Enemy"))
    {
        GameManager.instance.AddScore(1);  // +1 score for robot kill
        Destroy(other.gameObject);
    }
    Destroy(gameObject);  // Bullet always self destructs on any hit
}
```



6.3 Weighted Probability Spawner System

The Spawner acts as the **Game Master**, periodically invoking **`SpawnObject()`** on a random timer. A single **`Random.value`** float (0.0–1.0) is rolled and compared against cumulative thresholds: 20% crystal health packs, 30% road obstacles, 50% robot enemies selected from a prefab array (matching what is seen in Screenshot 3.2 two different robot sizes).

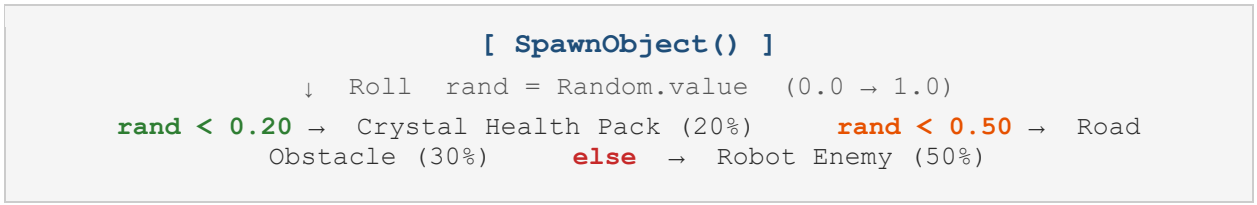


Figure: *Spawner weighted probability decision logic single random roll, cumulative threshold comparison.*

Listing 3 Spawner.cs: Weighted Random Spawning

```
// Spawner.cs   Weighted Random Spawning
void SpawnObject()
{
    int index = Random.Range(0, spawnPoints.Length);
    float rand = Random.value;  // Returns 0.0 to 1.0

    if (rand < healthSpawnChance && healthPackPrefab != null)
    {
        // 20%   Spawn Crystal Health Pack
        Instantiate(healthPackPrefab, spawnPoints[index].position,
Quaternion.identity);
    }
    else if (rand < healthSpawnChance + obstacleSpawnChance && obstaclePrefab !=
null)
    {
        // 30%   Spawn Road Obstacle
        Instantiate(obstaclePrefab, spawnPoints[index].position,
Quaternion.identity);
    }
    else if (enemyPrefabs.Length > 0)
    {
        // 50%   Spawn random Robot Enemy from prefab array
        int enemyIndex = Random.Range(0, enemyPrefabs.Length);
        Instantiate(enemyPrefabs[enemyIndex], spawnPoints[index].position,
Quaternion.identity);
    }
}
```

6.4 Progressive Difficulty Scaling

A global **`speedMultiplier`** float on the GameManager singleton increments by **`speedIncreaseRate × Time.deltaTime`** every frame, capped at **`maxSpeedMultiplier`**. Every moving object robot enemies, crystal health packs, road scroll reads this value at runtime, automatically inheriting the difficulty escalation without additional inter script coupling.

Listing 4 GameManager.cs & Obstacle.cs: Speed Escalation

```
// GameManager.cs   Global Speed Escalation
void Update()
```

```
{
    if (speedMultiplier < maxSpeedMultiplier)
    {
        speedMultiplier += speedIncreaseRate * Time.deltaTime;
    }
}

// Obstacle.cs   Consuming the Global Speed Multiplier
void Update()
{
    float currentSpeed = speed * GameManager.instance.speedMultiplier;
    transform.Translate(Vector2.left * currentSpeed * Time.deltaTime);
}
```

7. Creative Feature Implementations

Two polished creative features were implemented beyond the base assignment requirements. Both are focused on cinematic player feedback to heighten the emotional impact of taking damage in Robot Rush.

7.1 Screen Space Damage Flash Overlay

Upon losing a heart, a full screen **red UI Image overlay** is triggered. Rather than a simple opacity snap, the system uses **`Mathf.Lerp`** to smoothly animate the alpha transparency away over time. The baseline alpha is permanently raised as hearts decrease creating progressive visual urgency that communicates danger without a numeric display.

Table 4 Damage Flash Overlay Behaviour by Heart Count

HP State	Overlay Behaviour
3 Hearts (Full)	Flashes red on hit; fades to 0% opacity quickly via Mathf.Lerp
1 Heart (Critical)	Flash red on hit; permanent 50% red tint maintained as danger cue
0 Hearts (Dead)	Locks at 75% red sustained visual cue on Game Over screen

7.2 Camera Shake Coroutine

On collision with a robot or obstacle, a Unity **Coroutine** (**`IEnumerator`**) stores the camera's original **`Vector3`** position, then rapidly offsets X and Y via **`Random.Range( 1f, 1f) × magnitude`** each frame. Critically, it uses **`Time.unscaledDeltaTime`**, meaning the shake continues even when **`Time.timeScale = 0`** (Game Over pause state) leaving a lasting physical impression.

Listing 5 HealthSystem.cs: Camera Shake Coroutine

```
// HealthSystem.cs   Camera Shake Coroutine
IEnumerator ShakeCamera(float duration, float magnitude)
{
    Vector3 originalPos = mainCamera.transform.localPosition;
    float elapsed = 0f;

    while (elapsed < duration)
    {
        float x = Random.Range( 1f, 1f) * magnitude;
        float y = Random.Range( 1f, 1f) * magnitude;
        mainCamera.transform.localPosition = new Vector3(x, y, originalPos.z);
        elapsed += Time.unscaledDeltaTime;  // Works even when timeScale = 0
        yield return null;
    }
    mainCamera.transform.localPosition = originalPos;  // Restore camera
}
```

8. Assignment Requirements Compliance

The table below maps every SE4031 Assignment 02 requirement to its corresponding implementation in Robot Rush, confirming full rubric compliance.

Table 5 Requirements Traceability Matrix

#	Requirement	Implementation Summary	✓
1	**Health System**	Heart icon HP bar (3 hearts); TakeDamage() on robot collision; resets via GameManager.RestartGame()	✓
2	**Score System**	Score UI top right; enemy kill by bullet adds +1; score resets to 0 on restart	✓
3	**Health Packs**	Crystal pack collectable; Destroy() on contact; destroyed after timeout; capped at 3 hearts	✓
4	**Projectile Attacks**	Instantiate bullet at firePoint on MouseButton(0); Destroy on collision or lifetime expiry	✓
5	**Enemy Mechanics**	Robot prefabs; damage player on contact (lose heart); destroyed by bullet; +1 score on kill	✓
6	**Speed Scaling**	speedMultiplier increments each frame × Time.deltaTime; capped at maxSpeedMultiplier	✓
7	**Creative Feature**	TWO features: Damage Flash Overlay (Mathf.Lerp alpha) + Camera Shake Coroutine (IEnumerator)	✓✓

9. Credits & References

Game Engine: Unity 2022 LTS https://unity.com

Language: C# (Microsoft .NET / Mono Runtime)

IDE: Visual Studio / Visual Studio Code

Version Control: GitHub InteractiveMediaGD Classroom Repository

Assignment Creators: Mr. Aruna Ishara Gamage, Mr. Nushkan Nismi

Unity Documentation: https://docs.unity3d.com/Manual/Quickstart2DCreate.html

Screen Recording: OBS Studio / Windows Game Bar

Assets: All game assets (soldier sprite, robot prefabs, crystal packs, road background) created or sourced and modified by the student for this submission.

Submitted by: Dassanayake B.A. | IT22178954 | SE4031 – Games Development | March 2026